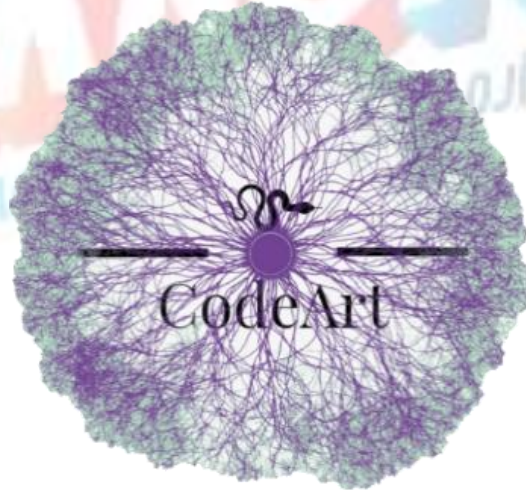


**TEKNOFEST
HAVACILIK, UZAY VE TEKNOLOJİ
FESTİVALİ**

**ULAŞIMDA YAPAY ZEKA YARIŞMASI
KRİTİK TASARIM RAPORU**

396835

CodeArt



İçindekiler

1. Takım Şeması.....	4
2. Poje Mevcut Durum Değerlendirmesi.....	4
3. Algoritmalar ve Sistem Mimarisi.....	4
3.1. Veri Setleri (10 Puan).....	4
3.2. Kullanılması Planlanan Algoritmalar	5
3.2.1. Faster R-CNN	5
3.2.2. YOLO.....	6
3.2.3. SSD.....	7
4. Özgünlük	8
4.1. Veri Arttırma (Data Augmentation)	8
4.2. Veri Çoğaltmanın Yaratabileceği Sorunlar ve Çözümleri	8
5. Sonuçlar ve İnceleme.....	9
6. Referanslar	11



Şekiller Listesi

Şekil 1. Takım organizasyonu ve görev dağılımı.....	4
Şekil 2. Veri setinden örnekler.....	5
Şekil 3. Faster R-CNN yazılım mimarisi	6
Şekil 4. YOLO mimarisi	6
Şekil 5. Aktivasyon fonksiyonlarının grafikleri	7
Şekil 6. SSD yazılım mimarisi	7
Şekil 7. VisDrone veri setinden alınan örnek görüntüler	8
Şekil 8. YOLOv4 yazılım mimarisi etiketler ve doğruluk oranları	10

Tablolar Listesi

Tablo 1. Veri setindeki sınıfların etiket sayıları.....	5
Tablo 2. Yazılım mimarilerinin avantaj ve dezavantajları	9
Tablo 3. YOLOv4 yazılım mimarisinin eğitim sonuçları.....	9
Tablo 4. YOLOv4 yazılım mimarisi kullanılarak yapılan eğitim sonucunda sınıflara ait doğruluk oranları	9
Tablo 5. Sınıflara ait TP ve FP sonuçları.....	10

Kısaltmalar

UAP : Uçan Araç Park

UAİ : Uçan Araç İniş

R-CNN : Region-based Convolutional Neural Networks

Faster R-CNN : Faster Region-based Convolutional Neural Networks

SSD : Single Shot Detector

YOLO : You Only Look Once

SVM : Support Vector Machines

TP : True Positive

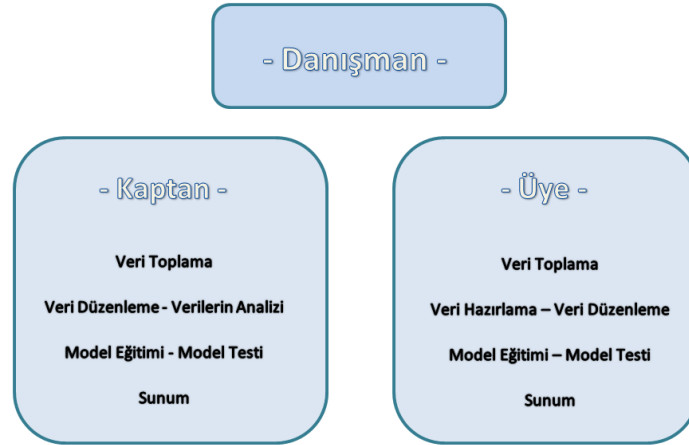
FP : False Positive

RPN : Feature Pyramid Network

FPS : Frames Per Second

IoU : Intersection of Units

1. Takım Şeması



Şekil 1. Takım organizasyonu ve görev dağılımı

Takımımıza, Yapay Zeka ve Veri Bilimi Uygulama ve Araştırma Merkezi müdürümüz danışmanlık yapmaktadır. Takımımız yeni kurulmuş olmakla birlikte deneyimli öğretim elemanlarıyla projeler yürütmektedir. Yapay öğrenme yolculuğuna doğal öğrenmeden başlamıştır. Yazılım alanını psikoloji, felsefe, mantık, antropoloji ve fizyoloji gibi birçok alanla birleştirerek eğitim görmüş ve çalışmalar yürütmüştür. Günümüz sorunlarına sanatsal çözümler üretme isteğiyle teknoloji döneminde yazılımla projeler geliştirmektedir. Takımımız ulaşım sektöründeki sorunlara da en iyi çözümler üretmek için Teknofest'in Ulaşım'da Yapay Zeka adlı yarışmasında yoğun ilgi ve çalışmasını sürdürecektir.

2. Proje Mevcut Durum Değerlendirmesi

Ön tasarım raporunda hakem heyeti tarafından geliştirilmesi istenen yönlerimiz veri setinin çoğaltılması ve daha iyi anlatılmasıdır. CodeArt olarak veri çoğaltma algoritmaları kullanarak veri setimizi genişlettik. Genişletilmiş veri seti ile daha kaliteli bir eğitim aşaması geçirerek yüksek doğruluk oranlarına sahip bir modeller elde ettik. Teknofest için bize verilmiş olan veri setinde UAP ve UAİ alanlarının yetersiz olmasında dolayı verilere Manuel Görsel Manipulasyonu uygulanmıştır [1][2]. Bu sayede yeterli sayıda UAP ve UAİ alanı elde edilmiştir. Veri setindeki bir diğer eksik ise yaya sayısıdır. Veri setimizde 25,378 adet yaya görseli bulunmaktadır. Bu sayı her ne kadar büyük görünse de yayalar görselde küçük bir alan kapladıkları için modelin yaya sınıfını öğrenmesi düşük kalmıştır. Ön tasarım raporundan sonraki süreçte kullanılması planlanan algoritmalar 3'e indirilmiştir. Belirlenen algoritmalar arasında en iyi sonuç veren algoritmalar YoloV4, Faster R-CNN ve SSD algoritmalarıdır.

3. Algoritmalar ve Sistem Mimarisi

3.1. Veri Setleri

Derin öğrenme mimarileri üzerinde yapılması planlanan eğitimlerde kullanılacak olan veri seti TC. Ulaştırma Bakanlığı ve T3 Vakfı tarafından yarışmada kullanılmak üzere Drive üzerinden paylaşılan 2022_Teknofest_Ulasimda_Yapay_Zeka_Yarismasi_Ornek_Video.mp4 isimli videodan saniyede 5 kare olacak şekilde fotoğraflara bölünmüştür. Videonun çerçevelere (framelere) bölünmesi ile elde edilen bu fotoğraflar kullanılması planlanan Denetimli Öğrenme (Supervised Learning) algoritmalarına uygun şekilde etiketlenerek eğitime hazır hale getirilmiştir. Ayrıca doğruluk oranını arttırabilmek için görüntü işleme fonksiyonları ile veri seti üzerinde düzenlemelerin yapılması düşünülmektedir. Doğruluk oranını olumlu yönde etkileyebilmek için başvuracağımız diğer bir yöntem ise dron yardımıyla resim ve video çekmektir. Dron yardımı hazırlanmış bir veri seti olan VisDrone eğitimlerimizde kullanılmıştır [3][4].

Tablo 1. Veri setindeki sınıfların etiket sayıları

Araç	Yaya	UAP	UAI
108,474	25,378	1,921	2,395



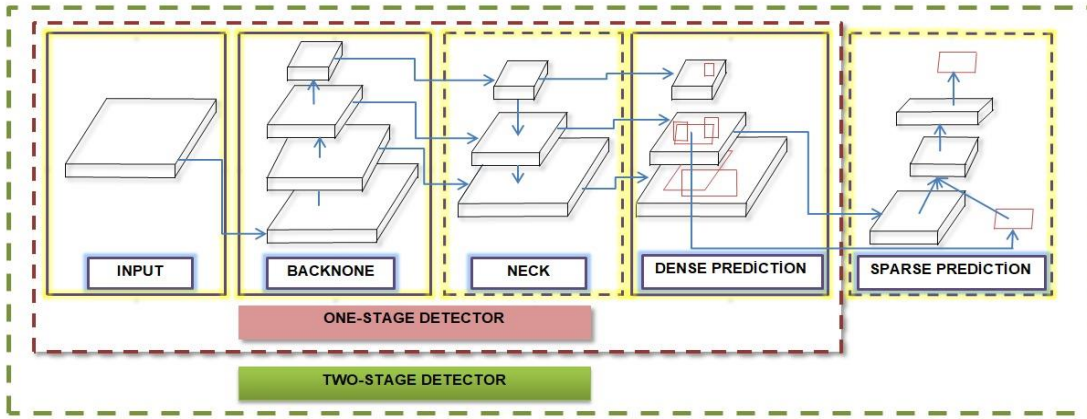
Şekil 2. Veri setinden örnekler

3.2. Kullanılması Planlanan Algoritmalar

Kullanılması planlanan algoritmalar nesne tespiti temelli derin öğrenme modelleri ile yaptığımız eğitim ve testlerde hem yüksek hız ve doğruluğa sahip olmaları hem en iyi (optimum) sonuç vermelerinden dolayı seçilmiştir. ÖTR sonrasında kullanılması kararlaştırılan 3 algoritma KTR teslim tarihine kadar olan süreçte denenmiştir.

3.2.1.Faster R-CNN

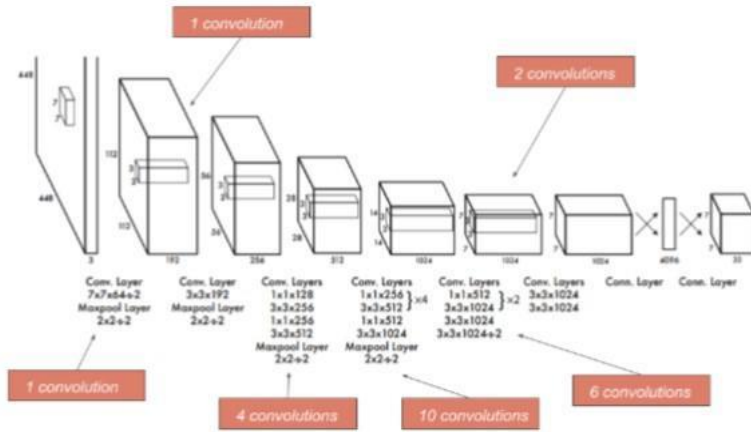
Faster R-CNN, grid benzeri verilerin işlenmesi için özel olarak geliştirilen CNN sinir ağı türü temel alınarak geliştirilmiştir. CNN ile nesneleri tespit etmek için resmin hangi bölümüne bakılması gerektiği bilinmelidir. Nesnelerin eksiksiz tespitini yapabilmek için resim çok küçük parçalara ayrılmalıdır ki bu da işlem süresini uzatmakta ve nesne olmayan bölümlerde de işlem yaparak gereksiz işlem yapılmasına yol açmaktadır [5]. Girshick ve ark. bu sorunu çözebilmek için R-CNN modelini geliştirmişlerdir [6]. R-CNN modeli resmi parçalara ayırma işlemini kayan pencere yöntemi olan Selective Search algoritmasıyla yapmaktadır. Selective Search algoritması nesne bulunma olasılığının yüksek olduğu bölgeleri hızlı bir şekilde tespit etmektedir, öğrenme içermeyen sabit bir algoritmadır [7]. Tespit edilen bölgelerin CNN kullanılarak özellikleri çıkarılmaktadır ve çıkarılan özellikler SVM ile sınıflandırılarak nesne tespit işlemi gerçekleştirilmektedir [5]. R-CNN geliştiricilerinden biri olan Grishick, R-CNN modelini eğitim süresi uzun olduğundan yeni bir model olan Fast R-CNN modelini geliştirmiştir [8]. Shaoqing ve ark. Selective Search, ağıın performansını etkileyen yavaş ve zaman alıcı bir işlem olduğundan ağıın bölge önerilerini öğrenmesini sağlayan bir nesne algılama algoritmasıyla nesne tespiti işlemini çok daha hızlandıran Faster R-CNN modelini geliştirmişlerdir [9]. YOLO, RetinaNet, EfficientNet ve SSD modelleri tek aşamalı nesne tespiti çalışmalarında ön plana çıkarken R-CNN, Fast R-CNN ve Faster R-CNN modelleri iki aşamalı nesne tespiti çalışmalarında başarı göstermektedir [10]. Benjdira ve ark. araç gözetimi ve trafik izleme senaryosunu gerçekleştirmek için tek aşamalı YOLOv3 ile iki aşamalı Faster R-CNN modelini karşılaştırmışlardır. İnsansız hava araçlarından alınan görüntülerle oluşturulmuş veri setini kullanarak yaptıkları çalışmada performans değerlendirmesini doğruluk ve işlem süresi açısından ele almışlardır. Çalışma sonuçlarına göre her iki algoritmada da kullanılan veri setinde nesne tahmin doğruluğu en az %99 ve üzerinde gerçekleşmiştir ama YOLOv3 algoritması, Faster R-CNN modeline göre daha sağlam ve daha yüksek hatırlamaya sahip olmuştur [11].



Şekil 3. Faster R-CNN yazılım mimarisi [12].

3.2.2.YOLO

YOLO (You Only Look Once / Sadece Bir Kez Bak), algılama ve sınıflandırmayı içeren tek adımlı bir süreç oluşturmak için geliştirilen modeldir [13]. YOLO, tek bir sinir ağı ile problemi uçtan uca iyileştirme yaparak doğrudan algılama performansına göre nesne tespiti yapmaktadır [14]. Özellikle son yıllarda nesne tespiti konusunda popüler olan algoritmalarından biridir. Nesne tespitinde hız önemli bir kavramdır [15]. YOLO'nun en hızlı mimarisi 45 FPS'ye ulaşabilir ve daha küçük bir sürümü olan Tiny-YOLO, GPU'lu bir bilgisayarda 244 FPS'ye (Tiny YOLO-V2) ulaşmaktadır [13].



Şekil 4. YOLO mimarisi

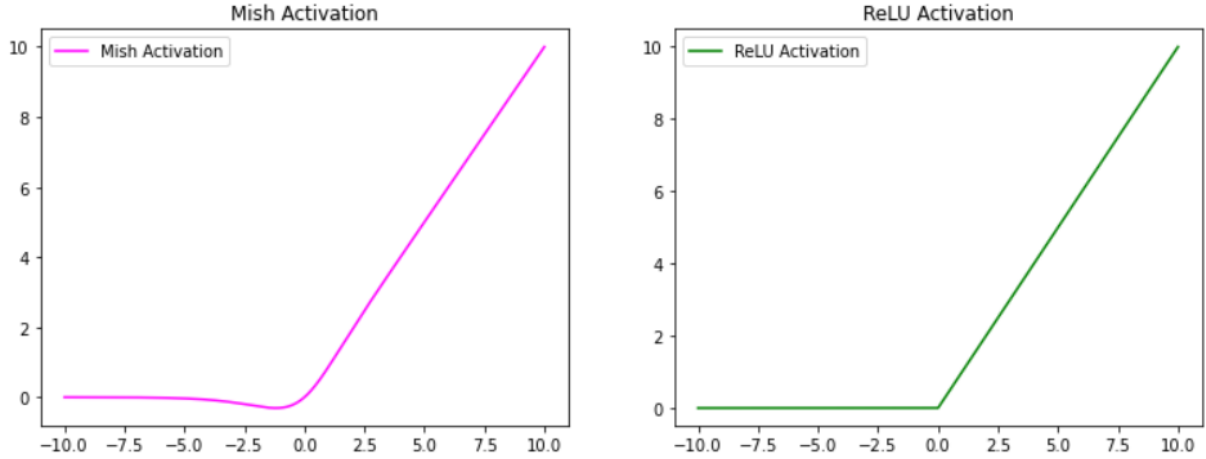
Model, özellik çıkarma için 24 evrişim katmanından ve ardından sınırlayıcı kutu koordinatlarını ve ilgili nesne olasılıklarını öngörmek için 2 tam bağlı katmandan oluşmaktadır [16].

YOLO'yu diğer algoritmalarından ayıran gerçek zamanlı nesne tespiti yapmasıdır [10]. YOLO hem performans hem başarı oranının yüksek olması nedeniyle gerçek zamanlı nesne tespiti çalışmalarında tercih edilmektedir [17].

YOLO mimarisi eğitim setinde bulunan nesnelerin konumlarını gösteren sınırlayıcı kutuları (bounding box) bulmak için giriş görüntüsünü $S \times S$ boyutunda ızgara (grid) şeklinde bölmüştür. Daha sonra her bir ızgara hücresi için bir nesne tanımı yapmıştır. Bir birine komşu ve aynı sınıfı gösteren ızgara hücresi birleştirilerek sınırlama kutusu elde etmiştir [18]. Algoritma, görüntüyü tek bir seferde sinir ağından geçirerek resimdeki tüm nesnelerin sınıfını ve koordinatlarını tahmin eder. Bu tahmin işleminin temeli, nesne tespitini tek bir regresyon problemi olarak ele almalarında yatmaktadır. Her bir ızgara kendi içinde, alanda nesnenin olup olmadığını, varsa orta noktasının içinde olup olmadığını, orta noktası da içindeyse uzunluğunu, yüksekliğini ve hangi sınıftan olduğunu bulmakla sorumludur. Buna göre YOLO her ızgara için ayrı bir tahmin vektörü oluşturur. Bu vektörlerin her birinin içinde

güven skoru, nesnenin orta noktasının x koordinatı, nesnenin orta noktasının y koordinatı, nesnenin genişliği, nesnenin yüksekliği, bağlı sınıf olasılığı, kutunun nesneyi kapsayıp kapsamadığının olasılığı (nesne var mı yok mu), IoU(ground truth ile tahmin edilmiş kutu arasındaki IoU değeri) bulunur [19].

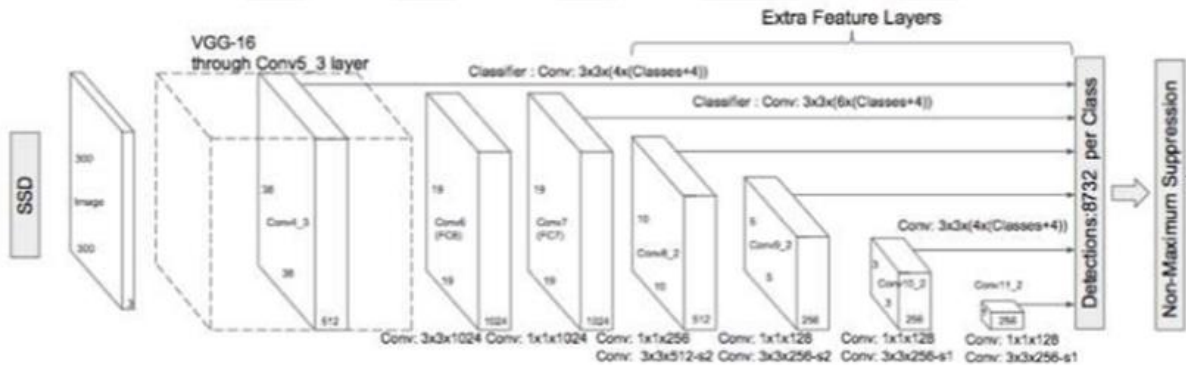
Bochkovskiy ve ark. tarafından 2020 yılında en uygun hız ve doğrulukla nesne tanıma yöntemi olarak YOLO-V4 sürümü çıkarılmıştır. Bu modelde giriş ağı çözünürlüğü, evrimsel katman sayısı, parametre sayısı ve katman çıktıların (filtreler) sayısı arasındaki en iyi dengeyi bulmak amaçlanmıştır [20]. YOLO-V4 mimarisinde “mish” isimli farklı bir aktivasyon fonksiyonunun kullanıldığı görülmektedir [21].



Şekil 5. Aktivasyon fonksiyonlarının grafikleri

3.2.3.SSD

SSD (Single Shot Multibox Detector), tek bir derin sinir ağı kullanarak tek seferde nesne tanıma işlemi yapmaktadır. Faster R-CNN'den daha hızlı çalışmaktadır çünkü diğer yöntemlerdeki eksiklikleri gidermek için SSD'de bazı iyileştirmeler yapılmıştır. SSD'nin daha düşük çözünürlüklü görüntüler kullanarak Faster R-CNN'nin doğruluk eşleştirmesini sağlayarak hızı yükseltir. SSD nesne tespiti özellik haritasının çıkarılması ve nesneleri algılamak için evrimci filtrelerinin uygulanması olmak üzere iki aşamada çalışmaktadır [22]. Projelerde hız önemliyse SSD kullanılmalı, daha doğru tespit önemliyse Faster R-CNN kullanılmalıdır. SSD sadece tek kez ileri gidiş yaptığından düşük donanımlı bir aygıtta da nesne sınıflandırma yapabilmektedir [23].



Şekil 6. SSD yazılım mimarisi

4. Özgünlük

Yarışma için verilen videodan elde edilen fotoğraflar ana veri setimizi oluşturmaktadır. Yaptığımız eğitim ve testlerin sonucuna bağlı olarak elimizdeki verinin her koşulda öğrenemediği görülmüştür. Bu sebepten dolayı ana veri setimize ek başka veri setleri bulunmuş, incelenmiş ve doğruluğu düşürecek görseller elenerek veri setine dahil edilmiştir. Alt görüş kamerası kullanan dronelar ve benzeri sistemler için hazırlanmış bir veri seti olan VisDrone, yarışma şartnamesinde bulunan şartlara uygun olduğundan dolayı veri setimize eklenmiştir [3].



Şekil 7. VisDrone veri setinden alınan örnek görüntüler

4.1. Veri Arttırma (Data Augmentation)

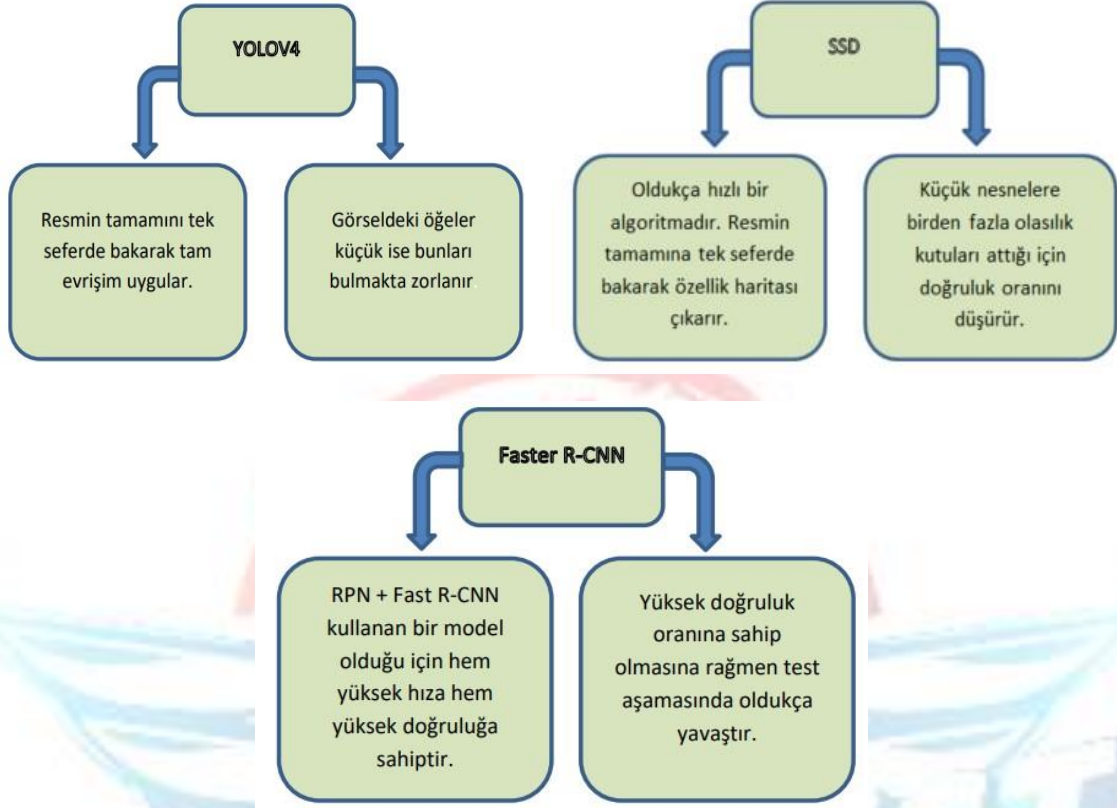
Veri arttırma (Data augmentation), eğitilecek olan derin öğrenme algoritmasının eğitim ve test için bölütleyeceği görsel sayısını arttırmak ve çeşitli çoğaltma teknikleri kullanarak modelin olabilecek en yüksek doğruluk oranına sahip olmasını sağlamak için uygulanan bir dizi görsel yönlendirme işlemidir. Veri setini çoğaltmak öğrenmenin her koşulda gerçekleştirilebilmesini sağlamak amacıyla kullanılmaktadır. Kullanılması planlanan veri çoğaltma teknikleri kernel filtreleri, rastgele silme, görsel birleştirme, renk uzayı dönüşümleri, düşmanca eğitim, çekişmeli üretici ağ, geometrik döndürme, kırpma, oklüzyon, yoğunluk işlemleri, gürültü enjeksiyonu, kombinasyon ve yakınlaştırmadır. Ön eğitilmiş ağların parametreleri ve katmanları değiştirilerek performans artırılması yapılmıştır [1][2][24][25][26].

4.2. Veri Çoğaltmanın Yaratabileceği Sorunlar ve Çözümleri

Veri setini çoğaltmak OverFitting'e yol açabilmektedir. Eğitilen modelin overfit olmasını engellemek amacıyla veri setinde uygulayacağımız yöntemler öznitelik sayısını azaltmak, veri çeşitliliğini arttırmak ve düzenlemektedir. Model eğitimi esnasında EarlyStop uygulanmıştır. Bu sayede modelin ezberlemesi engellenmiştir.

5. Sonular ve İnceleme

Tablo 2. Yazılım mimarilerinin avantaj ve dezavantajları



Algoritmalar ve Sistem Mimarisi başlığı altında açıklanan algoritmalar içinde en iyi sonuç veren algoritma YoloV4 algoritmasıdır. Bu nedenden dolayı YoloV4, yarışmada kullanılmak üzere seçilen nihai algoritmamız olmuştur. YoloV4 algoritması girdileri 416x416 boyutunda kabul eder. Model, 16.000 devir (epoch) sayısında tamamlanmıştır. Eğitim tamamlandıktan sonra elde edilen doğruluk, kayıp, doğrulama kaybı sonuçları Tablo 3' te belirtilmiştir.

Tablo 3. YOLOv4 yazılım mimarisinin eğitim sonuçları

Model	Doğruluk	Doğrulama Doğruluğu	Kayıp	Doğrulama Kaybı
YOLOv4	0.8321	0.8878	0.7294	0.7256

Tablo 4. YOLOv4 yazılım mimarisi kullanılarak yapılan eğitim sonucunda sınıflara ait doğruluk oranları

Ara	Yaya	UAP	UAI
%79.02	%75.56	%99.57	%99.83

Modelin değerlendirme kriterleri olarak karmaşıklık matrisi, duyarlılık, hassasiyet ve IoU kullanılacaktır. **Karışıklık Matrisi**, gerçek değerlerinin bilindiği bir dizi test verisi ile modelin tahmin değerlerinin karşılaştırılmasını sağlayan bir tablodur. **True Positive (Doğru Pozitif)**, verinin gerçek değeri 1 iken tahmin değerinin de 1 olarak gelmesidir. **False Positive (Yanlış Pozitif)**, verinin gerçek değeri 0 iken tahmin değerinin de 0 olarak gelmesidir.

Tablo 5. Sınıflara ait TP ve FP sonuçları

	Doğru Pozitif (TP)	Yanlış Pozitif (FP)
Araç	65353	22400
Yaya	25266	3021
UAP	649	24
UAI	690	26

Doğruluk, sınıfların ortalama hassasiyet (AP) değerlerinin aritmetik ortalaması (mAP) alınarak bulunmaktadır. Denklem 1’de açıklanan terimlere ait formüller verilmiştir.

$$AP = \left(\frac{1}{n} \right) \sum_{Recall_i} Precision(Recall_i) = 1$$

(1)

$$mAP = \frac{1}{|classes|} \sum_{ceclasses} \frac{\#TP(c)}{\#TP(c) + \#FP(c)}$$

Eğitilen modele ait tahmin etiketleri aşağıdadır.



Şekil 8. YOLOv4 yazılım mimarisi etiketler ve doğruluk oranları

6. Referanslar

- [1] "data-augmentation-for-custom-object-detection-15674966e0c8 @ medium.com." [Online]. Available: <https://medium.com/predict/data-augmentation-for-custom-object-detection-15674966e0c8>.
- [2] L. Nanni, S. Brahnam, and G. Maguolo, "Data Augmentation for Building an Ensemble of Convolutional Neural Networks," *Smart Innov. Syst. Technol.*, vol. 145, pp. 61–69, 2019, doi: 10.1007/978-981-13-8566-7_6.
- [3] "VisDrone-Dataset @ github.com." [Online]. Available: <https://github.com/VisDrone/VisDrone-Dataset>.
- [4] O. Sahin and S. Ozer, "YOLODrone: Improved YOLO Architecture for Object Detection in Drone Images."
- [5] B. Üniversitesi, "FBilimleriEnstitüsüElektrik -ElektronikMühendisliğiAnabilim Dalı GörüntülemeTeknikleri Kullanarak Petek," 2020.
- [6] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, pp. 580–587, 2014, doi: 10.1109/CVPR.2014.81.
- [7] J. R. R. Uijlings, K. E. A. Van De Sande, T. Gevers, and A. W. M. Smeulders, "Selective search for object recognition," *Int. J. Comput. Vis.*, vol. 104, no. 2, pp. 154–171, 2013, doi: 10.1007/s11263-013-0620-5.
- [8] R. Girshick, "Fast R-CNN," *Proc. IEEE Int. Conf. Comput. Vis.*, vol. 2015 International Conference on Computer Vision, ICCV 2015, pp. 1440–1448, 2015, doi: 10.1109/ICCV.2015.169.
- [9] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 6, pp. 1137–1149, 2017, doi: 10.1109/TPAMI.2016.2577031.
- [10] Salma, "No 主観的健康感を中心とした在宅高齢者における 健康関連指標に関する共分散構造分析Title," p. 6, 2021.
- [11] B. Benjdira, T. Khursheed, A. Koubaa, A. Ammar, and K. Ouni, "Car Detection using Unmanned Aerial Vehicles: Comparison between Faster R-CNN and YOLOv3," *2019 1st Int. Conf. Unmanned Veh. Syst. UVS 2019*, pp. 1–6, 2019, doi: 10.1109/UVS.2019.8658300.
- [12] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection." [Online]. Available: <https://github.com/AlexeyAB/darknet>.
- [13] R. Huang, J. Pedoeem, and C. Chen, "YOLO-LITE : A Real-Time Object Detection Algorithm Optimized for Non-GPU Computers," pp. 2503–2510, 2018.
- [14] F. G. Tan, A. S. Yüksel, E. Aydemir, and M. Ersoy, "Derin Öğrenme Teknikleri İle Nesne Tespiti Ve Takibi Üzerine Bir İnceleme A Review On Object Detection And Tracking With Deep Learning Techniques," no. 25, pp. 159–171, 2021, doi: 10.31590/ejosat.878552.
- [15] B. AKSOY, K. KORUCU, Ö. ÇALIŞKAN, Ş. OSMANBEY, and H. D. HALİS, "İnsansız Hava Aracı ile Görüntü İşleme ve Yapay Zekâ Teknikleri Kullanılarak Yangın Tespiti: Örnek Bir Uygulama," *Düzce Üniversitesi Bilim ve Teknol. Derg.*, vol. 9, pp. 112–122, 2021, doi: 10.29130/dubited.1016195.
- [16] F. Üniversitesi, "FEN BİLİMLERİ ENSTİTÜSÜ," 2017.
- [17] *EVRIŞİMLİ SİNİR AĞI ÖZELLİKLERİNE DAYANAN KORELASYON FİLTRELEME VE VERİ İLİŞKİLENDİRME İLE ÇOKLU NESNE TAKİBİ Elnura ARSLAN.*
- [18] A Buchari ·2018, 2018.

- [19] Ü. İstanbul *et al.*, “HAVACILIK , UZAY VE TEKNOLOJİ FESTİVALİ (ÖZGÜN ARAÇ KATEGORİSİ) TAKIM ADI : MİLAT 1453 Elektromobil Ar & Ge Topluluğu,” pp. 1–47.
- [20] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” 2020, [Online]. Available: <http://arxiv.org/abs/2004.10934>.
- [21] M. Peker *et al.*, “İnsan Sayma Sistemi için Gömülü Cihazlarda Derin Öğrenme Mimarilerinin Performans Analizi,” no. Imsec, pp. 1–10, 2020, [Online]. Available: <https://market.samm.com/raspberry-pi-4-4gb-en?src=raspberrypi>.
- [22] F. G. TAN, A. S. YÜKSEL, E. AYDEMİR, and M. ERSOY, “DERİN Öğrenme Teknikleriyle Nesnelerin Takibi Üzerine Bir İnceleme,” *Eur. J. Sci. Technol.*, no. 25, pp. 159–171, 2021, doi: 10.31590/ejosat.878552.
- [23] kue tradisional khas Aceh, “No Title E/AENH,” *Ayaz*, vol. 8, no. 5, p. 55, 2019.
- [24] H. Wang, Q. Wang, F. Yang, W. Zhang, and W. Zuo, “Data Augmentation for Object Detection via Progressive and Selective Instance-Switching,” Jun. 2019, doi: 10.48550/arxiv.1906.00358.
- [25] B. Zoph, E. D. Cubuk, G. Ghiasi, T. Y. Lin, J. Shlens, and Q. V. Le, “Learning Data Augmentation Strategies for Object Detection,” *Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics)*, vol. 12372 LNCS, pp. 566–583, 2020, doi: 10.1007/978-3-030-58583-9_34/TABLES/7.
- [26] E. D. Cubuk, B. Zoph, V. Vasudevan, and Q. V. Le Google Brain, “AutoAugment: Learning Augmentation Strategies from Data.” [Online]. Available: <https://pillow.readthedocs.io/en/5.1.x/>.

